

Informe de desarrollo: Comunicación Bluetooth entre ESP32 y pantalla LCD I2C mediante MIT App Inventor

1. Título del proyecto

Visualización de mensajes en una pantalla LCD I2C mediante comunicación Bluetooth con ESP32 y MIT App Inventor

2. Objetivo

El objetivo de la actividad fue implementar un sistema que permita enviar un mensaje escrito desde una aplicación móvil creada en **MIT App Inventor** hacia un microcontrolador **ESP32** mediante comunicación **Bluetooth**.

El mensaje recibido por el ESP32 debía mostrarse en una pantalla **LCD 16x2 con módulo I2C**.

3. Materiales utilizados

Para el desarrollo de la actividad se utilizaron los siguientes materiales:

- ESP32 DevKit V1.
 - Pantalla LCD 16x2 con módulo I2C.
 - Cables Dupont.
 - Cable USB para programar el ESP32.
 - Celular Android.
 - Plataforma MIT App Inventor.
 - Arduino IDE.
 - Librerías:
 - `BluetoothSerial`
 - `Wire`
 - `LiquidCrystal_I2C`
-

4. Descripción general del funcionamiento

El sistema funciona de la siguiente manera:

1. El usuario escribe un mensaje en una aplicación móvil creada en MIT App Inventor.
2. Al presionar un botón, la aplicación envía el mensaje por Bluetooth al ESP32.
3. El ESP32 recibe el texto enviado desde el celular.
4. Finalmente, el mensaje se muestra en la pantalla LCD I2C.

La pantalla LCD utilizada cuenta con un módulo I2C en la parte posterior, por lo que solo se emplearon cuatro pines de conexión:

- GND
- VCC
- SDA
- SCL

No fue necesario utilizar potenciómetro externo ni resistencia, ya que el módulo I2C de la pantalla incluye un potenciómetro integrado para regular el contraste.

5. Conexiones realizadas

Las conexiones entre el ESP32 y la pantalla LCD I2C fueron las siguientes:

Pantalla LCD I2C	ESP32 DevKit V1
GND	GND
VCC	5V / VIN
SDA	GPIO 21
SCL	GPIO 22

Se utilizó el pin **GPIO 21** del ESP32 para la línea **SDA** y el pin **GPIO 22** para la línea **SCL**, que son los pines comúnmente usados para comunicación I2C en el ESP32.

6. Programación del ESP32

Para programar el ESP32 se utilizó **Arduino IDE**. Primero se instalaron las librerías necesarias para controlar la pantalla LCD I2C y utilizar la comunicación Bluetooth.

La librería principal para la pantalla fue:

```
#include <LiquidCrystal_I2C.h>
```

También se utilizaron las librerías:

```
#include <BluetoothSerial.h>
#include <Wire.h>
```

El código cargado en el ESP32 permite:

- Iniciar el Bluetooth con el nombre `ESP32_LCD_BT`.
- Inicializar la pantalla LCD.
- Esperar mensajes enviados desde la aplicación móvil.

- Mostrar el mensaje recibido en la pantalla LCD.
- Dividir el mensaje en dos líneas si supera los 16 caracteres.

7. Código utilizado en el ESP32

```
#include <BluetoothSerial.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Bluetooth del ESP32
BluetoothSerial SerialBT;

// LCD I2C 16x2
// Direccion comun: 0x27
// Si no funciona, cambiar 0x27 por 0x3F
LiquidCrystal_I2C lcd(0x27, 16, 2);

String mensaje = "";

void setup() {
  Serial.begin(115200);

  // Iniciar I2C en ESP32
  // SDA = GPIO 21, SCL = GPIO 22
  Wire.begin(21, 22);

  // Iniciar LCD
  lcd.init();
  lcd.backlight();

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Esperando BT");
  lcd.setCursor(0, 1);
  lcd.print("ESP32_LCD_BT");

  // Iniciar Bluetooth
  SerialBT.begin("ESP32_LCD_BT");

  Serial.println("Bluetooth iniciado");
  Serial.println("Nombre: ESP32_LCD_BT");
}

void loop() {
  // Si llega un mensaje desde App Inventor por Bluetooth
  if (SerialBT.available()) {
    mensaje = SerialBT.readStringUntil('\n');
    mensaje.trim();
  }
}
```

```

Serial.print("Mensaje recibido: ");
Serial.println(mensaje);

lcd.clear();

// Si el mensaje tiene 16 caracteres o menos
if (mensaje.length() <= 16) {
  lcd.setCursor(0, 0);
  lcd.print(mensaje);
}
// Si el mensaje tiene más de 16 caracteres
else {
  lcd.setCursor(0, 0);
  lcd.print(mensaje.substring(0, 16));

  lcd.setCursor(0, 1);
  lcd.print(mensaje.substring(16, 32));
}
}
}

```

8. Desarrollo de la aplicación en MIT App Inventor

En MIT App Inventor se creó una aplicación sencilla con los siguientes componentes:

Componente	Función
Label	Mostrar el título de la aplicación
ListPicker	Seleccionar el dispositivo Bluetooth
TextBox	Escribir el mensaje que se desea enviar
Button	Enviar el mensaje al ESP32
Label	Mostrar el estado de conexión
BluetoothClient	Permitir la comunicación Bluetooth

La aplicación permite seleccionar el dispositivo Bluetooth previamente emparejado, conectarse al ESP32 y enviar el texto escrito en el cuadro de texto.

9. Componentes usados en MIT App Inventor

Los componentes usados fueron:

- **Label1** : título de la aplicación.
- **ListPicker1** : botón para seleccionar el Bluetooth.
- **TextBox1** : cuadro donde se escribe el mensaje.

- `Button1` : botón para enviar el mensaje.
- `Label2` : muestra el estado de la conexión.
- `BluetoothClient1` : componente no visible para la comunicación Bluetooth.

10. Bloques implementados en MIT App Inventor

Se realizaron tres bloques principales.

10.1. Bloque para mostrar dispositivos Bluetooth

Este bloque permite que, antes de seleccionar un dispositivo, se cargue la lista de dispositivos Bluetooth disponibles y emparejados en el celular.

```
when ListPicker1.BeforePicking
  set ListPicker1.Elements to BluetoothClient1.AddressesAndNames
```

10.2. Bloque para conectar con el ESP32

Este bloque permite conectarse al dispositivo Bluetooth seleccionado. Si la conexión es exitosa, se muestra el mensaje **"Estado: conectado"**. Si no se logra conectar, se muestra un mensaje de error.

```
when ListPicker1.AfterPicking
  if call BluetoothClient1.Connect address ListPicker1.Selection
  then
    set Label2.Text to "Estado: conectado"
  else
    set Label2.Text to "No se pudo conectar"
```

10.3. Bloque para enviar el mensaje

Este bloque permite enviar el texto escrito en el TextBox hacia el ESP32. Se agregó el salto de línea `\n` al final del mensaje, ya que el código del ESP32 lee el texto hasta encontrar ese carácter.

```
when Button1.Click
  if BluetoothClient1.IsConnected
  then
    call BluetoothClient1.SendText text join TextBox1.Text "\n"
    set Label2.Text to "Mensaje enviado"
  else
    set Label2.Text to "Primero conecta Bluetooth"
```

11. Pruebas realizadas

Después de cargar el programa en el ESP32, se abrió el Monitor Serial a **115200 baudios**.

Se verificó que el ESP32 iniciara correctamente mostrando el mensaje:

```
Bluetooth iniciado  
Nombre: ESP32_LCD_BT
```

Luego se revisó que la pantalla LCD encendiera y mostrara el mensaje inicial:

```
Esperando BT  
ESP32_LCD_BT
```

Después, se probó la aplicación móvil siguiendo estos pasos:

1. Se encendió el ESP32.
2. Se abrió la aplicación creada en MIT App Inventor.
3. Se presionó el botón para conectar Bluetooth.
4. Se seleccionó el dispositivo correspondiente al ESP32.
5. Se escribió un mensaje en el cuadro de texto.
6. Se presionó el botón de enviar.
7. El mensaje enviado se mostró en la pantalla LCD.

12. Observaciones

Durante el desarrollo se identificó que la pantalla LCD tenía un módulo I2C integrado, por lo que no era necesario realizar la conexión tradicional usando los pines:

- RS
- RW
- E
- D4
- D5
- D6
- D7

Esto permitió simplificar el circuito, utilizando únicamente cuatro cables.

También se observó que el contraste de la pantalla puede regularse usando el pequeño potenciómetro integrado en el módulo I2C de la parte posterior de la pantalla.

En caso de que la pantalla no muestre texto, se puede probar cambiando la dirección I2C de:

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

por:

```
LiquidCrystal_I2C lcd(0x3F, 16, 2);
```

13. Problemas encontrados

Durante las pruebas se observó que el ESP32 mostraba en el Monitor Serial un mensaje relacionado con el arranque del sistema:

```
Core dump data check failed
```

Sin embargo, el programa continuó ejecutándose correctamente, ya que luego apareció el mensaje:

```
Bluetooth iniciado  
Nombre: ESP32_LCD_BT
```

Por lo tanto, se concluyó que el ESP32 estaba funcionando correctamente y que el Bluetooth se había iniciado.

También se consideró que, si el celular no detecta el dispositivo Bluetooth, se debe verificar lo siguiente:

- Que el ESP32 esté encendido.
 - Que el programa esté cargado correctamente.
 - Que el celular sea Android.
 - Que se esté usando un ESP32 compatible con Bluetooth clásico.
 - Que el Bluetooth del celular esté activado.
 - Que el ESP32 se busque desde los ajustes Bluetooth del celular antes de usar la app.
-

14. Conclusión

Se logró implementar un sistema de comunicación Bluetooth entre una aplicación móvil creada en MIT App Inventor y un ESP32.

El mensaje escrito desde el celular fue enviado correctamente al microcontrolador y mostrado en una pantalla LCD 16x2 con módulo I2C.

Esta actividad permitió comprender el uso de la comunicación Bluetooth en el ESP32, el manejo de pantallas LCD mediante I2C y la integración entre hardware y una aplicación móvil desarrollada de manera visual en MIT App Inventor.